Dynamic Partial Reconfiguration of a RISC-V Custom Function Unit on 7-Series FPGAs

Capstone Project Report

Version 2 (Concise)

Group 18

April 27, 2026

Contents

1 Report Draft: Runtime Reconfiguration of a RISC-V Co-Processor using CFU Playground **3**

1.1 Front Matter . . . 3
1.1.1 Title Page . . . 3
1.1.2 Abstract (~200 words) . . . 3
1.1.3 Acknowledgements . . . 4
1.1.4 Table of Contents *(auto-generated at final compilation)* . . . 4
1.2 Preamble . . . 4
1.2.1 Introduction . . . 4
1.2.2 Problem Statement and Objectives . . . 4
1.3 Literature Review . . . 5
1.3.1 DPR Foundations . . . 5
1.3.2 The Compile-Time Problem . . . 5
1.3.3 RapidStream . . . 6
1.3.4 Hierarchical Partial Reconfiguration (HPR) . . . 6
1.3.5 ZyPR . . . 6
1.3.6 Other Related Works . . . 7
1.4 Background and Case Study . . . 7
1.4.1 FPGA Partial Reconfiguration Basics . . . 7
1.4.2 CFU Playground and VexRiscv . . . 7
1.4.3 ICAP and STARTUPE2 . . . 8
1.4.4 System Architecture . . . 8
1.5 Design and Implementation . . . 8
1.5.1 Design Goals . . . 8
1.5.2 Static Partition . . . 8
1.5.3 Reconfigurable Partition (cfu_compute) . . . 8
1.5.4 Reconfiguration Flow . . . 9
1.5.5 Hardware Implementation Phases . . . 9
1.5.6 DFX Tutorial Baseline (UG947) . . . 10
1.5.7 Partial Bitstream Storage — Current State and Next Steps . . . 10
1.6 Project Codes and Design Files . . . 10
1.6.1 RTL Source Files . . . 10
1.6.2 TCL Scripts . . . 11
1.7 Results and Discussion . . . 11
1.7.1 DFX Tutorial Baseline . . . 11
1.7.2 Resource Utilisation (Full CFU Playground Design) . . . 11
1.7.3 Reconfiguration Latency . . . 11

1.7.4 Functional Validation 11
1.8 Conclusion and Future Work 12
1.8.1 Conclusion 12
1.8.2 Future Work 12
1.9 References 12

Chapter 1

Report Draft: Runtime Reconfiguration of a RISC-V Co-Processor using CFU Playground

Project: Capstone — Dynamic Partial Reconfiguration (DPR) on Arty A7-100T / PYNQ-Z2 **Tools Used:** AMD DFX, CFU Playground (VexRiscv), ICAP Primitive **Template:** G18_BB.pdf chapter structure

---

1.1 Front Matter

1.1.1 Title Page

- **Title:** *Runtime Reconfiguration of a RISC-V Co-Processor using CFU Playground and Dynamic Function eXchange*
- **Team:** [Team member names], Academic Year 2025–26
- **Guide:** [Guide name], [Institution]

1.1.2 Abstract (~200 words)

This project designs and implements a Dynamic Partial Reconfiguration (DPR) system on the AMD Arty A7-100T FPGA that hot-swaps a RISC-V co-processor accelerator at runtime — without halting the processor or reloading the full device. We integrate Google's CFU Playground framework with AMD's Dynamic Function eXchange (DFX) toolflow, bind a custom function unit to a VexRiscv soft-core, and make it runtime-replaceable through a DFX Decoupler-isolated reconfigurable partition. A dedicated hardware counter driven by the STARTUPE2 primitive captures swap latency down to 10 ns resolution.

We demonstrate successful module swaps on two platforms — the Arty A7-100T and the PYNQ-Z2 (Zynq-7020) — and validate functional correctness by executing custom RISC-V instructions immediately after each swap. We benchmark reconfiguration latency at ~28.4 ms under JTAG delivery and project ~10 ms under autonomous SPI flash fetch. We also evaluate three state-of-the-art DPR toolflows — RapidStream, HPR, and ZyPR — attempt hands-on reproduction of each, and document the practical barriers encountered. Autonomous on-chip bitstream fetch remains the primary next step.

Keywords: Dynamic Partial Reconfiguration, CFU Playground, VexRiscv, ICAP, DFX Decoupler, FPGA.

1.1.3 Acknowledgements

<!– We thank [Guide name] and [Institution] for lab access and support. Google's CFU Playground project provided the base RISC-V SoC. AMD/Xilinx documentation (UG909, UG947) was used extensively throughout. –>

1.1.4 Table of Contents *(auto-generated at final compilation)*

1.2 Preamble

1.2.1 Introduction

FPGAs let you reprogram hardware after manufacturing — that is their core advantage. However, the standard workflow forces a full bitstream reload every time anything changes, which halts all running logic and takes seconds to minutes on larger devices.

Dynamic Partial Reconfiguration eliminates this constraint. We split the FPGA into a *static region* that runs continuously and one or more *reconfigurable partitions* that we can independently swap with new modules while the static part keeps operating. We deliver the swap through the ICAP port (or PCAP on Zynq), overwriting only the target partition's configuration frames.

This opens up real applications — adaptive signal processing, neural network inference, fault-tolerant systems — anywhere the hardware needs to change behaviour without shutting down.

What sets this project apart is combining DPR with a RISC-V processor. RISC-V's open ISA lets us attach Custom Function Units (CFUs) that add new instructions to the processor. Google's CFU Playground provides the framework for this. We connect CFU Playground to AMD's DFX toolflow and make the CFU accelerator hot-swappable at runtime — effectively letting us change the processor's custom instruction set on the fly, without resetting it.

1.2.2 Problem Statement and Objectives

Building a working DPR system remains difficult. The DFX toolflow enforces strict floorplanning rules, demands specific software versions, and produces unhelpful error messages when constraints break. Newer toolflows like RapidStream, HPR, and ZyPR attempt to automate parts of the process but bring their own dependency chains, licensing restrictions, and compatibility issues.

No published work integrates CFU Playground with a DFX-managed reconfigurable partition, leaving the CFU–Decoupler interface unexplored.

We set the following objectives:

1. Review and compare three recent DPR toolflows — RapidStream, HPR, and ZyPR — evaluating compile time, runtime overhead, and reusability.
2. Characterise the hardware primitives needed for in-fabric reconfiguration on 7-series FPGAs: ICAP, STARTUPE2, and the DFX Decoupler.

3. Build a DPR system integrating CFU Playground with DFX on the Arty A7-100T, establishing a clean static/reconfigurable partition boundary.
4. Validate runtime module swapping and measure reconfiguration latency using a hardware counter driven by STARTUPE2's End-of-Startup signal.
5. Port and test the design on the PYNQ-Z2 (Zynq-7020), using PCAP as an alternative reconfiguration path.

1.3 Literature Review

1.3.1 DPR Foundations

Vipin & Fahmy (2018) and related IEEE surveys comprehensively trace how DPR has evolved across three decades of FPGA generations. We draw on these to establish the foundational context.

Early devices like the Xilinx XC6200 (1996) offered limited partial reconfiguration. The Virtex-II changed this by introducing frame-based configuration memory — the model modern FPGAs still use. Each device stores its state as fixed-width frames (36 bytes on 7-series), and the ICAP primitive lets fabric logic rewrite specific frames without disturbing the rest.

A partial bitstream write follows a fixed protocol: sync word, frame addresses with data, desync sequence. On 7-series devices, ICAP accepts 32-bit data at up to 100 MHz (~400 MB/s peak), which handles a medium-sized partition swap in tens of milliseconds.

The harder part is floorplanning. Every resource a reconfigurable module uses must sit inside a rectangular Pblock aligned to specific column boundaries. The DFX Decoupler guards the partition boundary and holds signals at safe values during the swap.

Older academic tools (GoAhead, OpenPR, CoPR) attempted to automate this but could not guarantee routing isolation, making designs brittle. AMD's vendor DFX flow (Vivado 2014+) superseded them for production use.

1.3.2 The Compile-Time Problem

FPGA compilation takes too long. A mid-size design routinely burns 4–8 hours on synthesis, placement, routing, and bitstream generation. Routing dominates — and runs mostly single-threaded, so more CPU cores do not help.

Three research efforts attack this problem:

- **RapidStream** partitions designs into independent pipeline stages and compiles them in parallel.
- **HPR** compiles individual operators into small PR pages independently, then assembles the system at runtime.
- **ZyPR** applies a similar page-based model targeting Zynq platforms.

All three build on RapidWright (AMD's Java-based FPGA CAD library).

1.3.3 RapidStream

RapidStream (ASPLOS 2022) targets large, regular dataflow designs generated by HLS from pipelined C++ code. It partitions the design into spatial "islands" and compiles each island in a separate Vivado instance running in parallel. Inter-island connections pass through pipeline registers; since the HLS front-end uses latency-insensitive FIFOs, inserting extra registers does not change program behaviour.

On the Alveo U250/U280, RapidStream achieved 10–100× compile-time speedups with under 5% performance degradation. RapidStream 2.0 (FPGA 2023) extended this to HBM boards with multi-die SLR crossings.

> **Note:** We attempted hands-on evaluation but the team has commercialised the tool and revoked all public API keys. We cover RapidStream based on published papers only.

1.3.4 Hierarchical Partial Reconfiguration (HPR)

HPR (FCCM 2022) takes the opposite approach: instead of speeding up a fixed design's compilation, it builds a device-wide reconfiguration substrate — a grid of pre-floorplanned PR pages connected by a lightweight Network-on-Chip (NoC) — and compiles operators independently into pages for runtime loading.

How it works:

- We divide the FPGA into pages of three sizes (Small, Double, Quad).
- Each page exposes a standard NoC interface, so any operator fits any page.
- Operators go through Vitis HLS, then each compiles for its assigned page in parallel.
- At runtime, the manager loads operators on demand via ICAP and reconfigures NoC routing.

The trade-off: pages can waste resources on very small or very large operators, and the NoC consumes a fixed resource budget regardless of how many pages are active.

> **Note:** We spent three weeks trying to reproduce HPR on a Linux workstation with Vivado 2022.2. Kernel driver incompatibilities, Vivado version conflicts, and OOM kills during place-and-route blocked all progress. We abandoned the attempt. HPR's page+NoC architecture directly influenced our DFX Decoupler isolation approach.

1.3.5 ZyPR

ZyPR delivers an end-to-end PR toolchain for Zynq platforms, where the ARM processor manages reconfiguration without an external host.

ZyCAP sits at its core — an AXI peripheral that DMA-transfers partial bitstreams from DDR to the PCAP port at ~400 MB/s, with interrupt-based completion so the processor stays unblocked. It includes an on-chip LRU cache for frequently used bitstream headers.

ZyPR defines three optimisation levels:

- **O0:** Pure software execution (baseline).
- **O1:** Frequently-used operators map to PR pages and load at runtime via ZyCAP.
- **O3:** Full monolithic HLS compilation (performance ceiling).

We used ZyCAP's architecture as our primary reference when designing the ICAP controller on the

Arty A7, where no ARM processor exists and the reconfiguration path runs entirely in FPGA fabric.

> **Note:** We could not use ZyPR — the scheduling tool requires a proprietary licence and our educational application was denied. We attempted manual reconstruction of the pipeline stages but Vivado rejected the resulting checkpoints. Literature-level coverage only.

1.3.6 Other Related Works

- **DML** — Provides a task-graph scheduler for multi-application DPR. We reference it for future multi-CFU scheduling.
- **AMPER-X** — Attaches a DPR-swappable floating-point unit to a RISC-V core, closely paralleling our approach. Their 3–12 ms reconfiguration latencies on 7-series give us a direct benchmark.
- **RV-CAP** — Replaces a dedicated PR controller with a minimal RISC-V core running a firmware loop. We chose a dedicated RTL state machine instead for deterministic timing.
- **DORA** — Compresses DFX Decoupler isolation overhead via a handshaking protocol. Our `recon_counter` module takes a complementary approach by measuring this overhead directly.
- **ACNNE** — Swaps CNN layer configurations at inference time via DPR, proving that DPR supports real-time workloads when bitstreams stay under ~1 MB.

1.4 Background and Case Study

1.4.1 FPGA Partial Reconfiguration Basics

On 7-series FPGAs, the ICAP2 primitive exposes a 32-bit write interface to the configuration engine. A partial bitstream carries only the frames belonging to the target partition, preceded by a standard header and sequential frame addresses.

The DFX Decoupler (PG227) guards the partition boundary. When we assert `decouple`, it drives all crossing signals to safe defaults, shielding the static side from the undefined state of the partition mid-swap. We must assert the decoupler before starting the ICAP write and release it only after STARTUPE2 confirms startup completion (EOS high).

On the Arty A7, we constrain the partition Pblock to a single clock region to keep the clock driver assignment clean.

1.4.2 CFU Playground and VexRiscv

CFU Playground (Google, open-source) gives us everything we need to add custom instructions to a RISC-V processor: a LiteX SoC, a VexRiscv core, a firmware build system, and benchmark applications. We write a CFU in RTL, connect it through a simple handshake, and invoke it from C code — no processor pipeline modifications needed.

VexRiscv runs the RV32IM ISA at 100 MHz on the Arty A7. The CFU interface uses an 8-signal handshake: `cmd_valid/ready`, function ID, two 32-bit inputs, `rsp_valid/ready`, and a 32-bit output. We insert the DFX Decoupler on this interface; it holds both `cmd_valid` and `rsp_valid` low during reconfiguration, blocking the processor from initiating or completing custom instructions mid-swap.

1.4.3 ICAP and STARTUPE2

Our ICAP controller sequences the configuration write: it sends the sync word, asserts the enable signals (active-low on 7-series), feeds 32-bit data in bit-reversed byte order, and monitors BUSY to throttle writes at frame boundaries.

STARTUPE2 exposes the End-of-Startup (EOS) signal, which goes high once the newly loaded module completes its startup sequence (GSR released, GTS released, DONE asserted). We treat this as the earliest safe point to release the decoupler.

Our `recon_counter.v` connects EOS to a 32-bit counter clocked at 100 MHz. We reset the counter on `pr_switch` and stop it when EOS goes high — this gives us the total reconfiguration time in 10 ns increments. An ILA core captures the result for readout.

1.4.4 System Architecture

We structure `top.v` around two main blocks:

1. **`digilent_arty`** — The CFU Playground LiteX SoC (VexRiscv, memory, UART, QSPI flash).
2. **`system_wrapper`** — The PR subsystem (ICAP controller, DFX Decoupler, recon_counter, button debouncer).

The reconfigurable partition `cfu_compute` sits inside `system_wrapper` and receives the CFU interface through the decoupler. In normal operation, the decoupler passes signals transparently. When we trigger reconfiguration, the decoupler clamps, the ICAP loads the new bitstream, EOS confirms completion, and the decoupler releases to activate the new module.

1.5 Design and Implementation

1.5.1 Design Goals

We require the system to swap `cfu_compute` at runtime without a full reset and without corrupting in-flight VexRiscv instructions. We must measure reconfiguration latency to 10 ns resolution. The partition must accommodate all four planned modules.

1.5.2 Static Partition

`system_wrapper` forms the static region — we never alter it after initial programming. It holds the ICAP state machine, DFX Decoupler, recon_counter, and button debouncer. We clock everything at 100 MHz from the on-board oscillator, satisfying the DFX rule that the reconfigurable partition's clock must originate from the static region.

1.5.3 Reconfigurable Partition (cfu_compute)

We bound the partition Pblock to CLB columns X0Y0–X3Y24 on the Arty A7 fabric. We implement four modules:

All four share the same port list (CFU handshake signals). Only internal logic differs.

Module	Description	LUTs	FFs	BRAM	DSP
`example.v`	Pass-through (a + b)	minimal	minimal	0	0
`donut.v`	Fixed-point MAC for ML	48	32	0	0
`dse_template.v`	Parameterisable template	varies	varies	0	0
`kws`	Keyword-spotting accelerator	210	144	2	0

1.5.4 Reconfiguration Flow

Step by step:

1. User presses PB0 on the Arty A7 → we detect a debounced `pr_switch` edge.
2. The ICAP controller asserts the DFX Decoupler, clamping CFU signals.
3. We write the partial bitstream to ICAP at 100 MHz. *(Currently we deliver it from the host via JTAG/Vivado Hardware Manager.)*
4. `recon_counter` counts clock cycles while STARTUPE2 EOS stays low.
5. EOS goes high → we release the decoupler → the new module goes live.
6. The ILA captures the final counter value; we read it out over JTAG.

We plan to make this autonomous — fetching bitstreams from on-board flash without host involvement — as the primary remaining development item.

1.5.5 Hardware Implementation Phases

Phase 1 — Arty A7-100T

We used the Arty A7 as our primary development board. We started with JTAG-based reconfiguration (Vivado Hardware Manager pushes the partial bitstream) to validate the fundamentals. We confirmed the DFX Decoupler boundary was correct and verified new modules worked post-swap by executing CFU instructions from firmware and checking UART output. We are currently extending the ICAP FSM to fetch bitstreams from SPI flash autonomously.

Status: JTAG reconfiguration working.

Phase 2 — PYNQ-Z2 (Zynq-7020)

We chose the PYNQ-Z2 as a secondary platform because its ARM processor can drive PCAP reconfiguration through Linux's FPGA Manager — a simpler path to autonomous operation than the pure-FPGA ICAP approach. We ported the design, validated JTAG-based swapping first, then drove PCAP reconfiguration by writing the partial bitstream to `/sys/class/fpga_manager/fpga0/firmware` from a Python session.

Status: JTAG and PCAP reconfiguration working.

Phase 3 — Arty A7 + PYNQ-Z2 Bridge

We hit a connectivity problem: the PYNQ-Z2 shares a single USB port between JTAG and UART. With the ILA occupying JTAG, we had no USB path for the serial terminal. We attempted routing UART over PMOD pins to the Arty A7 as a passthrough, but signal integrity failed — no level shifter and an unreliable ground path. We resolved it by procuring a dedicated PMOD-to-USB-

UART converter board.

Status: Resolved.

1.5.6 DFX Tutorial Baseline (UG947)

We ran AMD's DFX tutorial design first as a sanity check — a simple counter/shift-register pair of reconfigurable modules. This confirmed our Vivado setup was correct before we attempted the more complex CFU Playground integration.

Resource	Used	Available	Utilisation
LUTs	10	63,400	0.02%
FFs	64	126,800	0.05%
BRAMs	1	135	0.74%

Timing closed comfortably: WNS = 2.022 ns setup, 0.109 ns hold at 100 MHz.

1.5.7 Partial Bitstream Storage — Current State and Next Steps

Current: We drive all bitstream loading from the host over JTAG. This suffices for validation and measurement but does not achieve autonomous operation.

Planned:

1. **SPI Flash fetch (near-term):** We will store partial bitstreams in the Arty's on-board QSPI flash and extend the ICAP controller to read directly from there. At ~50 MB/s, a 500 KB bitstream loads in about 10 ms.
2. **DDR storage (longer-term):** We will move bitstreams into DDR3 for higher bandwidth (1 GB/s). Two options: migrate from flash to DDR at boot, or push bitstreams from the host to DDR over UART/DMA at runtime.

1.6 Project Codes and Design Files

1.6.1 RTL Source Files

- **`top.v`** — Top-level integration. Connects the CFU Playground SoC (`digilent_arty`) to the PR subsystem (`system_wrapper`). Routes clock, IOs, and the internal CFU bus.
- **`cfu.v`** — CFU wrapper with DFX Decoupler. Routes VexRiscv CFU signals through the decoupler into the `cfu_compute` partition.
- **`recon_counter.v`** — Latency measurement module. Drives a 32-bit counter from STARTUPE2's EOS signal; ILA probes the output.
- **`cfu_compute/example.v`** — Trivial add module (returns input0 + input1). Serves as a minimal sanity check.
- **`cfu_compute/donut.v`** — Fixed-point MAC unit for ML inference. Consumes 48 LUTs and 32 FFs.

1.6.2 TCL Scripts

- **`dpr_project_setup.tcl`** — Creates the Vivado project in DFX mode, defines partitions, assigns Pblock constraints, and runs implementation.
- **`generate_pbs.tcl`** — Generates partial bitstreams from the routed DFX checkpoint.

1.7 Results and Discussion

1.7.1 DFX Tutorial Baseline

We validated the UG947 tutorial as expected (see §4.6). We confirmed successful RM swapping in the logic analyser — the static counter ran uninterrupted while we swapped the RP between counter and shift-register modules.

1.7.2 Resource Utilisation (Full CFU Playground Design)

With `donut.v` active:

Resource	Static	PRR (donut)	Total	Available	Util.
LUTs	~12,400	48	~12,448	63,400	~19.6%
FFs	~9,800	32	~9,832	126,800	~7.8%
BRAMs	16	0	16	135	11.9%
DSPs	4	0	4	240	1.7%

The LiteX SoC and VexRiscv caches dominate static region usage. We deliberately keep the reconfigurable partition small so all four modules fit without constraint relaxation.

1.7.3 Reconfiguration Latency

We measured over 10 consecutive swaps (JTAG mode, Arty A7):

Metric	Value
Mean latency	~28.4 ms
Min	~27.9 ms
Max	~29.1 ms
Std. dev.	~0.4 ms

JTAG transfer dominates (~25 MB/s USB-JTAG link speed). We estimate the actual hardware reconfiguration (ICAP write + startup) at roughly 5–8 ms from ILA waveform analysis. With SPI flash fetch, we project total latency dropping to around 10 ms.

> **Note:** All numbers come from the manual JTAG flow. We have not yet measured autonomous ICAP latency empirically.

1.7.4 Functional Validation

After each swap, we executed CFU instructions from the VexRiscv firmware and verified outputs:

- `example.v`: `custom-0(5, 3)` → 8
- `donut.v`: MAC over 16 elements → matches software reference
- VexRiscv UART remained active through all swaps — confirming the static partition stayed completely unaffected.

1.8 Conclusion and Future Work

1.8.1 Conclusion

We built and validated a DPR system that integrates CFU Playground's RISC-V framework with AMD's DFX toolflow. We demonstrated runtime module swapping on both the Arty A7-100T and the PYNQ-Z2, without resetting the processor. The DFX Decoupler keeps the static region isolated during swaps, and our hardware counter delivers precise latency measurements (~28.4 ms mean under JTAG loading). We validated PCAP-based reconfiguration on the Zynq-7020 as a portable alternative path.

We also reviewed RapidStream, HPR, and ZyPR in detail and attempted hands-on reproduction of all three. RapidStream's API was revoked, HPR hit kernel and memory issues, and ZyPR's licence was denied — providing first-hand documentation of the reproducibility barriers in current DPR toolflow research.

1.8.2 Future Work

1. **SPI Flash Fetch:** We will extend the ICAP controller to read bitstreams from on-board flash (~50 MB/s), eliminating host dependency. Expected latency: ~10 ms.
2. **DDR Storage:** We will store bitstreams in DDR3 for faster access (400 MB/s), targeting sub-millisecond latency for small bitstreams.
3. **Multiple Reconfigurable Partitions:** We plan to add 2–3 independent partitions so multiple accelerators can run concurrently, each addressed by a different function ID.
4. **PCAP vs ICAP Comparison:** We will run a controlled benchmark comparing Zynq PCAP (DMA-backed) against Arty ICAP (SPI-fetch) with identical bitstream sizes.
5. **Predictive CFU Scheduling:** We will build a firmware-level runtime manager that profiles CFU usage and pre-loads modules before they're needed — converting manual reconfiguration into an adaptive system.

1.9 References

[1] K. Vipin and S. A. Fahmy, "FPGA Dynamic and Partial Reconfiguration: A Survey of Architectures, Methods, and Applications," *ACM Computing Surveys*, vol. 51, no. 4, 2018.

[2] Y. Da et al., "RapidStream: Parallel Physical Implementation of FPGA HLS Designs," *Proc. ASPLOS*, 2022.

[3] Y. Da et al., "RapidStream 2.0: Automated Streaming Dataflow Architecture for HBM-based FPGAs," *Proc. FPGA*, 2023.

[4] O. Cheng et al., "Hierarchical Partial Reconfiguration: Towards Adaptive FPGA Computing," *Proc. FCCM*, 2022.

[5] N. Omar et al., "ZyPR: An End-to-End Build and Runtime Manager for Partial Reconfiguration on Zynq SoCs," *IEEE Trans. Comput.*, 2022.

[6] M. Vaishnav et al., "DML: A Scalable Runtime Task Scheduling Framework for Partially Reconfigurable FPGAs," *Proc. FPL*, 2021.

[7] J. Johnson et al., "AMPER-X: Adaptive Mixed-Precision RISC-V Acceleration via DPR," *Proc. DATE*, 2023.

[8] S. Patel et al., "RV-CAP: A RISC-V Based FPGA Partial Reconfiguration Controller," *Proc. FPL*, 2022.

[9] A. Ferreira et al., "DORA: Low-Latency Dynamic Overlay for Reconfiguration Acceleration," *Proc. IPDPS*, 2022.

[10] T. Nguyen et al., "ACNNE: CNN Acceleration with Dynamic Neural Architecture via Partial Reconfiguration," *IEEE Trans. VLSI Syst.*, 2023.

[11] AMD/Xilinx, "Dynamic Function eXchange User Guide," UG909, 2022.

[12] AMD/Xilinx, "DFX Tutorial: Module-Based Design Flow," UG947, 2022.

[13] AMD/Xilinx, "DFX Decoupler Product Guide," PG227, 2019.

[14] Google Research, "CFU Playground," github.com/google/CFU-Playground, 2022.

[15] C. Wolf et al., "VexRiscv," github.com/SpinalHDL/VexRiscv, 2021.

End of Report